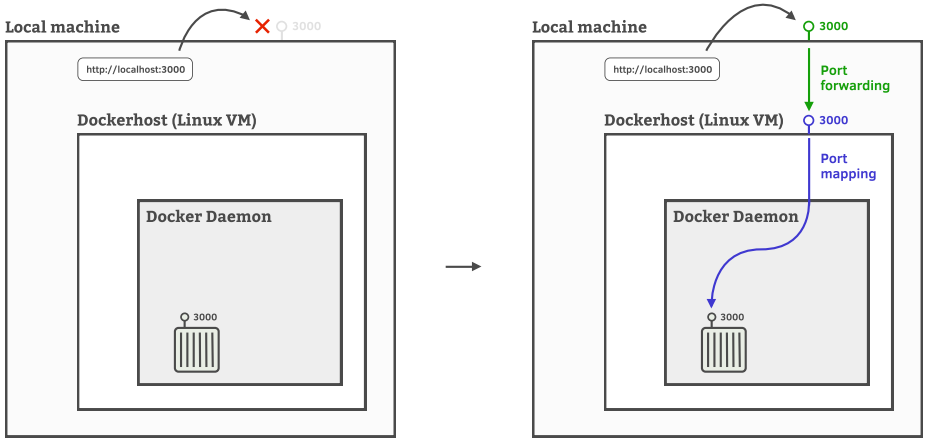


When publishing a port, you don't have to use the same external port as the service inside the container. If we had specified -p 80:3000, it would have mapped port 80 on our local machine to the Rails server listening on port 3000 inside the container. This gives us a lot of flexibility in terms of how we expose services to the outside world.

Binding the Rails Server to IP Addresses

Normally, to start a Rails server, we'd simply run bin/rails s, yet when we started the Rails server with docker run, we used bin/rails s -b 0.0.0.0. Why was that?

When you start the Rails server with bin/rails s, by default, it only listens to requests on localhost (or 127.0.0.1) on whatever machine it's running on. This provides a secure default, preventing the Rails app server from being accessible externally. However, in our case, the server is running inside a container, but the request is coming from outside.

When we request http://localhost:3000 on our local machine, as we've just seen in Reaching the App: Publishing Ports, on page 26, the request is forwarded to the Docker Engine. This in turn routes the request to the container running the Rails server, by translating the request to [IP address of container]:3000. However, as the Rails server is only listening to requests on localhost, nothing responds to this request coming in for a different IP address.

To fix this, we have to tell our Rails server to *bind* to *all* IP addresses, not just to localhost, using the option -b 0.0.0.0. The IP address 0.0.0.0 is a special address that means *"all IPv4 address on this machine."*